`ScalikeJdbcSetup.init(system)`. You see this in the `Main` of the application, but before that, let's create the final projection.

15.1.2 Kafka projection

This projection is used to publish the market events in a Kafka topic. It can be used to react to the `Market.Close` event to settle all bets related to that market and pay the winners or, for example, notify multiple clients that a market has opened or closed so they can react accordingly. For example, a notification can be sent to app users that a new market is available to bet.

Essentially, this projection is like the previous one, except that the handler writes to Kafka instead of a database. This difference is divided into two parts: processing the message in the handler, and the settings for the Kafka producer.

The Kafka producer has producer settings like any other producer you learned about in chapter 14. It uses a string serializer for the key and a byte array serializer for the value. The rest of the configuration is read from the betting-house project's application.conf file.

However, you are not in an Akka Stream here, so you can't just use a producer like `Producer.committableSink`. But you can use something you haven't seen before—a `SendProducer` that requires producer settings and an `ActorSystem`, as shown here:

```
package akka.kafka.scaladsl

final class SendProducer[K, V] private (
               val settings: ProducerSettings[K, V],
               system: ActorSystem)
```